
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

Web Technology

BCA — Semester 3 — 2020 — Answer Sheet

Subject Code: CACS205

Group B

Attempt any SIX questions. [6×5=30]

2. What is HTML5? Explain characteristics of HTML5.

[5]

Answer: HTML5: HTML5 is the fifth and current major version of the Hypertext Markup Language (HTML), the standard markup language used for structuring and presenting content on the World Wide Web. It was developed by the W3C (World Wide Web Consortium) and WHATWG (Web Hypertext Application Technology Working Group) and became a W3C Recommendation in October 2014. HTML5 introduces new elements, attributes, and behaviors, and provides a larger set of technologies that allows building diverse and powerful Web sites and applications.

Characteristics of HTML5:

1. **Semantic Elements:** HTML5 introduces semantic tags that clearly describe their meaning to both the browser and the developer. Examples include `<header>`, `<footer>`, `<article>`, `<section>`, `<nav>`, `<aside>`, and `<main>`. These improve accessibility and SEO.
2. **Multimedia Support:** Native support for audio and video through `<audio>` and `<video>` tags without requiring third-party plugins like Flash. The `<canvas>` element enables dynamic, scriptable rendering of 2D shapes and bitmap images.
3. **Graphics and Effects:** The `<canvas>` element allows drawing graphics via JavaScript. SVG (Scalable Vector Graphics) is also supported natively for resolution-independent graphics.
4. **Form Enhancements:** New input types such as email, url, date, time, number, range, color, and search. New form attributes like placeholder, required, pattern, autofocus, and autocomplete improve user experience and validation.
5. **Geolocation API:** Allows websites to access the user's geographical location with permission, enabling location-based services.
6. **Local Storage:** HTML5 provides web storage mechanisms — `localStorage` (persistent) and `sessionStorage` (per-session) — allowing websites to store data locally within the user's browser, replacing cookies for larger storage needs.
7. **Offline Web Applications:** The Application Cache (AppCache) and Service Workers enable web applications to work offline by caching resources locally.
8. **Drag and Drop:** Native API for dragging and dropping elements within a webpage, enabling intuitive user interfaces.
9. **Web Workers:** Allows JavaScript to run in background threads, improving performance for computation-intensive tasks without blocking the UI.
10. **Cross-Document Messaging:** The `postMessage` API enables secure communication between documents from different origins (cross-origin messaging).
11. **Improved Code:** HTML5 provides cleaner, more consistent code with a simplified DOCTYPE (`<!DOCTYPE html>`) and charset declaration (`<meta charset="UTF-8">`).

3. Write HTML tag to generate the following table.

[5]

Answer: The table shows Average Red Eyes data for Males and Females with Weight and Eyes percentages.

HTML Code:

```
<!DOCTYPE html> <html> <head> <title>Average Red Eyes Table</title> <style> table { border-collapse: collapse; width: 50%; margin: 20px auto; } th, td { border: 1px solid black; padding: 10px; text-align: center; } th { background-color: #f2f2f2; } </style> </head> <body> <table> <tr> <th rowspan="2">Average</th> <th colspan="2">Red Eyes</th> </tr> <tr> <th>Weight</th> <th>Eyes</th> </tr> <tr> <td>Males</td> <td>1.45%</td> <td>40%</td> </tr> <tr> <td>Females</td> <td>1.47%</td> <td>43%</td> </tr> </table> </body> </html>
```

Explanation:

- The <table> tag creates the table structure.
- <tr> defines each table row.
- <th> defines header cells with rowspan="2" for the "Average" cell spanning two rows, and colspan="2" for the "Red Eyes" header spanning two columns.
- <td> defines standard data cells for Males and Females rows with their respective values.
- CSS styling is applied for borders, padding, and background colors to make the table visually appealing.

4. Design the following website's Menu with use of HTML list and appropriate CSS.

[5]

Answer: Design a horizontal website menu with items: Home | News | Contact | About using HTML list and CSS.

HTML and CSS Code:

```
<!DOCTYPE html> <html> <head> <title>Website Menu</title> <style> * { margin: 0; padding: 0; box-sizing: border-box; } body { font-family: Arial, sans-serif; } nav { background-color: #333; padding: 0; } nav ul { list-style-type: none; overflow: hidden; display: flex; justify-content: center; } nav ul li { display: inline-block; } nav ul li a { display: block; color: white; text-align: center; padding: 14px 20px; text-decoration: none; font-size: 16px; transition: background-color 0.3s; } nav ul li a:hover { background-color: #555; color: #ffd700; } nav ul li a.active { background-color: #4CAF50; } </style> </head> <body> <nav> <ul> <li><a href="#home" class="active">Home</a></li> <li><a href="#news">News</a></li> <li><a href="#contact">Contact</a></li> <li><a href="#about">About</a></li> </ul> </nav> </body> </html>
```

Explanation:

- (unordered list) creates the menu structure with (list items) for each menu option.
- list-style-type: none removes default bullet points.
- display: flex and justify-content: center create a horizontal, centered menu.
- The <a> tags inside make each item clickable.
- padding: 14px 20px provides comfortable spacing around menu items.
- text-decoration: none removes underlines from links.
- :hover pseudo-class changes background and text color when mouse hovers.
- .active class highlights the current page with a different background color.

5. Compare and contrast between a block-level element and an inline element in HTML?

[5]

Answer: Block-Level Elements vs Inline Elements in HTML:

Aspect	Block-Level Elements	Inline Elements
Display Behavior	Always start on a new line and take up the full width available	Do not start on a new line and only take up as much width as necessary
Width	Occupy 100% of parent container width by default	Occupy only the space required by their content
Height	Height can be set explicitly; stacks vertically	Height and width cannot be set directly (ignored in standard HTML)
Margin & Padding	Respect all margin and padding properties (top, bottom, left, right)	Respect left and right margins/padding; top and bottom may not affect layout
Nesting	Can contain both block-level and inline elements	Can only contain data and other inline elements (not block-level)
Examples	<code><div></code> , <code><p></code> , <code><h1></code> - <code><h6></code> , <code></code> , <code></code> , <code><table></code> , <code><form></code> , <code><header></code> , <code><footer></code> , <code><section></code>	<code></code> , <code><a></code> , <code></code> , <code></code> , <code></code> , <code><label></code> , <code><input></code> , <code>
</code> , <code><code></code>
CSS Display	<code>display: block</code> (default)	<code>display: inline</code> (default)
Use Case	Structural layout, containers, sections, paragraphs	Styling text within content, links, emphasis, inline images

Example Illustrating Both:

```
<div style="background-color: yellow;"> This is a block-level div. It takes full width. </div> <p>This is another block element.</p> <span style="background-color: lightblue;"> This is inline </span> <a href="#" style="background-color: lightgreen;"> This link is also inline </a> <span> — they sit side by side.</span>
```

Note: CSS provides `display: inline-block` which combines characteristics of both — the element is inline (sits in text flow) but can have block-level properties like width, height, and vertical margins.

6. What is CSS Selector? Explain different types of selector with suitable example.

[1+4]

Answer: CSS Selector: A CSS selector is a pattern used to select and target HTML elements that you want to style. Selectors are the bridge between your HTML structure and CSS rules. When a browser parses CSS, it uses selectors to determine which elements should receive the declared styles. CSS selectors can target elements by name, ID, class, attribute, state, or relationship to other elements.

Types of CSS Selectors with Examples:

1. **Universal Selector (*):** Selects all elements on the page.

* { margin: 0; padding: 0; }

2. **Element/Type Selector:** Selects all elements of a specific type.

p { color: blue; font-size: 16px; }

3. **Class Selector (.):** Selects elements with a specific class attribute. Can be used multiple times.

.highlight { background-color: yellow; } // HTML: <p class="highlight">Important text</p>

4. **ID Selector (#):** Selects a single element with a specific ID. Must be unique per page.

#header { background-color: #333; color: white; } // HTML: <div id="header">Welcome</div>

5. **Descendant Selector (space):** Selects elements that are descendants of a specified element.

div p { color: red; } // All <p> inside any <div> }

6. **Child Selector (>):** Selects direct children only.

ul > li { list-style-type: square; } // Only direct children of }

7. **Adjacent Sibling Selector (+):** Selects an element immediately preceded by a specified element.

h2 + p { font-style: italic; } // First <p> immediately after <h2> }

8. **General Sibling Selector (~):** Selects all siblings preceded by a specified element.

h2 ~ p { color: gray; } // All <p> siblings after <h2> }

9. **Attribute Selector:** Selects elements based on an attribute or attribute value.

input[type="text"] { border: 1px solid blue; } a[href^="https"] { color: green; } // Links starting with https }

10. **Pseudo-class Selector (:):** Selects elements in a specific state.

a:hover { color: red; } input:focus { border-color: orange; } li:nth-child(odd) { background-color: #f0f0f0; }

11. **Pseudo-element Selector (::):** Selects specific parts of an element.

p::first-line { font-weight: bold; } p::after { content: " ✓"; color: green; }

7. Why session is required in web development? Explain how you set and remove values stored in Session with suitable example?

[1+4]

Answer: Why Session is Required in Web Development:

HTTP is a stateless protocol, meaning each request from a client to a server is independent and the server does not retain any information about previous requests. Sessions are required in web development to maintain state and persist user-specific data across multiple page requests. Key reasons for using sessions include:

1. **User Authentication:** Sessions keep users logged in as they navigate between pages.
2. **Shopping Carts:** E-commerce sites use sessions to remember items added to a cart.
3. **Personalization:** Remembering user preferences like language, theme, or settings.
4. **Form Data:** Preserving multi-step form data across pages.
5. **Security:** Storing sensitive information server-side rather than in cookies.

Setting and Removing Session Values (PHP Example):

Setting Session Values:

```
<?php // Start the session (must be first before any output) session_start(); // Set session variables
$_SESSION["username"] = "ram123"; $_SESSION["email"] = "ram@example.com";
$_SESSION["login_time"] = date("Y-m-d H:i:s"); echo "Session values set successfully!<br>"; echo "Welcome, " . $_SESSION["username"]; ?>
```

Accessing Session Values:

```
<?php session_start(); if (isset($_SESSION["username"])) { echo "User: " . $_SESSION["username"] . "<br>";
echo "Email: " . $_SESSION["email"]; } else { echo "No session data found."; } ?>
```

Removing Session Values:

```
<?php session_start(); // Remove a single session variable unset($_SESSION["email"]); echo "Email removed
from session.<br>"; // Remove all session variables $_SESSION = array(); echo "All session variables
cleared.<br>"; // Destroy the session completely session_destroy(); echo "Session destroyed."; ?>
```

Logout Example:

```
<?php session_start(); $_SESSION = array(); // Also delete the session cookie if
(isset($_COOKIE[session_name()])) { setcookie(session_name(), "", time()-3600, '/'); } session_destroy();
header("Location: login.php"); exit(); ?>
```

Explanation:

- session_start() must be called at the beginning of every PHP page that uses sessions.
- \$_SESSION is a superglobal array used to store session data.
- unset() removes a specific session variable.
- \$_SESSION = array() clears all session data.
- session_destroy() destroys the session file on the server.
- The session ID is typically passed via a cookie, which should also be cleared during logout.

8. Critically analysis the pitfall of the 3-tier technology in comparison with n-tier technology.

[5]

Answer: Critical Analysis: Pitfalls of 3-Tier Technology vs N-Tier Technology

3-Tier Architecture: Consists of Presentation Tier (UI), Application Tier (Business Logic), and Data Tier (Database).

N-Tier Architecture: Extends 3-tier by further decomposing tiers into more specialized layers (e.g., separating business logic into service layer, domain layer, validation layer, caching layer, etc.).

Pitfalls of 3-Tier compared to N-Tier:

1. Limited Scalability:

In 3-tier, the application tier handles all business logic, which can become a bottleneck as user load increases. N-tier distributes functionality across multiple specialized servers, allowing independent scaling of specific components (e.g., scaling only the caching layer without scaling the entire application tier).

2. Tight Coupling:

3-tier tends to bundle many responsibilities into a single application tier, leading to tight coupling between different business functions. N-tier promotes loose coupling by separating concerns into distinct, independent services that communicate through well-defined APIs.

3. Poor Maintainability:

As applications grow, the monolithic application tier in 3-tier becomes complex and difficult to maintain. A change in one business rule may affect unrelated functionality. N-tier isolates changes to specific layers, reducing the risk of unintended side effects.

4. Technology Stack Constraints:

In 3-tier, all business logic typically uses the same technology stack. N-tier allows different tiers to use the most appropriate technology for their specific purpose (e.g., Node.js for API gateway, Python for data processing, Java for business rules).

5. Deployment Challenges:

Deploying updates in 3-tier often requires redeploying the entire application tier, causing downtime. N-tier enables independent deployment of individual services, supporting continuous delivery and reducing deployment risk.

6. Fault Isolation:

In 3-tier, a failure in one part of the application tier can bring down the entire tier. N-tier provides better fault isolation — if the notification service fails, other services continue functioning.

When 3-Tier is Still Appropriate:

- Small to medium applications with straightforward requirements
- Projects with limited budget and development resources
- Applications where simplicity is preferred over flexibility
- Teams without microservices or distributed systems expertise

Conclusion: While 3-tier is simpler to design and implement initially, it struggles with the demands of large, complex, evolving applications. N-tier (and its modern evolution, microservices) provides superior scalability,

maintainability, and flexibility at the cost of increased complexity in deployment, monitoring, and inter-service communication.

Group C

Attempt any TWO questions. [2×10=20]

9. Design the following layout of webpage with use of div and appropriate CSS.

[10]

Answer: Design a webpage layout with Logo, Navigation, Header Banner, Side Bar, Body Area, and Footer sections using div and CSS.

HTML Code:

```
<!DOCTYPE html> <html> <head> <title>Webpage Layout</title> <style> * { margin: 0; padding: 0; box-sizing: border-box; } body { font-family: Arial, sans-serif; } .container { width: 100%; max-width: 1200px; margin: 0 auto; } #logo { background-color: #2c3e50; color: white; padding: 20px; text-align: left; font-size: 24px; font-weight: bold; } #navigation { background-color: #34495e; padding: 15px; } #navigation ul { list-style: none; display: flex; } #navigation ul li { margin-right: 20px; } #navigation ul li a { color: white; text-decoration: none; font-size: 16px; } #navigation ul li a:hover { color: #3498db; } #header-banner { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); color: white; padding: 60px 20px; text-align: center; font-size: 28px; } .content-wrapper { display: flex; min-height: 400px; } #sidebar { width: 25%; background-color: #ecf0f1; padding: 20px; } #sidebar h3 { margin-bottom: 15px; color: #2c3e50; } #sidebar ul { list-style: none; } #sidebar ul li { padding: 10px 0; border-bottom: 1px solid #bdc3c7; } #body-area { width: 75%; padding: 20px; background-color: #ffffff; } #body-area h2 { color: #2c3e50; margin-bottom: 15px; } #body-area p { line-height: 1.6; color: #555; } #footer { background-color: #2c3e50; color: white; text-align: center; padding: 20px; } </style> </head> <body> <div class="container"> <div id="logo">MyWebsite Logo</div> <div id="navigation"> <ul> <li><a href="#">Home</a></li> <li><a href="#">About</a></li> <li><a href="#">Services</a></li> <li><a href="#">Contact</a></li> </ul> </div> <div id="header-banner"> Welcome to Our Website </div> <div class="content-wrapper"> <div id="sidebar"> <h3>Sidebar</h3> <ul> <li>Category 1</li> <li>Category 2</li> <li>Category 3</li> <li>Category 4</li> </ul> </div> <div id="body-area"> <h2>Body Area</h2> <p>This is the main content area of the webpage. Here you can place articles, blog posts, product information, or any other primary content. The layout uses CSS Flexbox to create a responsive two-column design with a sidebar and main body area.</p> </div> </div> <div id="footer"> &copy; 2020 MyWebsite. All rights reserved. </div> </div> </body> </html>
```

Explanation:

- Each section (logo, navigation, header-banner, sidebar, body-area, footer) is enclosed in a <div> with a unique ID.
- The container div centers the layout and sets a maximum width.
- display: flex on content-wrapper creates the two-column layout for sidebar and body area.
- width: 25% for sidebar and width: 75% for body area divides the space proportionally.
- CSS styling provides colors, padding, and responsive behavior for each section.

10. Write a server-side script for login process assume that user name and password have already exist on database TU under the user table (id, user, password).

[10]

Answer: Server-Side Login Script (PHP with MySQL):

Database Setup:

```
CREATE DATABASE TU; USE TU; CREATE TABLE user ( id INT AUTO_INCREMENT PRIMARY KEY, user VARCHAR(50) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL ); -- Insert sample users (passwords should be hashed in production) INSERT INTO user (user, password) VALUES ('admin', MD5('admin123')), ('student1', MD5('pass123')), ('ram', MD5('ram2020'));
```

Login Form (login.html):

```
<!DOCTYPE html> <html> <head> <title>Login</title> <style> body { font-family: Arial, sans-serif; background-color: #f4f4f4; display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; } .login-box { background: white; padding: 40px; border-radius: 8px; box-shadow: 0 0 10px rgba(0,0,0,0.1); width: 350px; } .login-box h2 { text-align: center; color: #333; margin-bottom: 30px; } .form-group { margin-bottom: 20px; } .form-group label { display: block; margin-bottom: 5px; color: #555; } .form-group input { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-size: 14px; } .form-group input[type="submit"] { background-color: #4CAF50; color: white; border: none; cursor: pointer; font-size: 16px; padding: 12px; } .form-group input[type="submit"]:hover { background-color: #45a049; } .links { text-align: center; margin-top: 15px; } .links a { color: #4CAF50; text-decoration: none; font-size: 14px; } .error { color: red; text-align: center; margin-bottom: 15px; } </style> </head> <body> <div class="login-box"> <h2>Login Form</h2> <form action="login.php" method="POST"> <div class="form-group"> <label>User Name</label> <input type="text" name="username" required> </div> <div class="form-group"> <label>Password</label> <input type="password" name="password" required> </div> <div class="form-group"> <input type="submit" name="login" value="Login"> </div> <div class="links"> <a href="#">Lost your password?</a><br><br> <a href="#">Don't have an account? Signup here!</a> </div> </form> </div> </body> </html>
```

Login Processing Script (login.php):

```
<?php session_start(); // Database connection $host = "localhost"; $dbuser = "root"; $dbpass = ""; $dbname = "TU"; $conn = new mysqli($host, $dbuser, $dbpass, $dbname); if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); } if (isset($_POST['login'])) { $username = $conn->real_escape_string($_POST['username']); $password = md5($_POST['password']); // MD5 for demo; use password_hash() in production $sql = "SELECT * FROM user WHERE user='$username' AND password='$password'"; $result = $conn->query($sql); if ($result->num_rows == 1) { $row = $result->fetch_assoc(); $_SESSION['user_id'] = $row['id']; $_SESSION['username'] = $row['user']; echo "<script>alert('Login Successful!');</script>"; echo "Welcome, " . $_SESSION['username'] . "!"; // header("Location: dashboard.php"); } else { echo "<script>alert('Invalid username or password!');</script>"; echo "<a href='login.html'>Go Back</a>"; } } $conn->close(); ?>
```

Secure Version with Prepared Statements (login_secure.php):

```
<?php session_start(); $conn = new mysqli("localhost", "root", "", "TU"); if (isset($_POST['login'])) { $stmt = $conn->prepare("SELECT id, user FROM user WHERE user = ? AND password = MD5(?)"); $stmt->bind_param("ss", $_POST['username'], $_POST['password']); $stmt->execute(); $result = $stmt->get_result(); if ($result->num_rows == 1) { $user = $result->fetch_assoc(); $_SESSION['user_id'] = $user['id']; $_SESSION['username'] = $user['user']; header("Location: dashboard.php"); exit(); } else { header("Location: login.html?error=invalid"); exit(); } } ?>
```

Explanation:

- The login form collects username and password via POST method.
- login.php connects to the TU database and queries the user table.
- Passwords are compared using MD5 hashing (for demonstration; production should use password_hash() and password_verify()).

-
- On successful login, session variables are set and user is redirected.
 - The secure version uses prepared statements to prevent SQL injection attacks.

11. What are the rules for Well-formed xml document? Create a sample well-formed xml and validate it with DTD Schema.

[5+5]

Answer: Rules for Well-Formed XML Document:

A well-formed XML document must follow these syntactic rules defined by the W3C XML specification:

1. **Prolog:** The document should begin with an XML declaration (optional but recommended): `<?xml version="1.0" encoding="UTF-8"?>`
2. **Single Root Element:** There must be exactly one root element that contains all other elements.
3. **Proper Nesting:** All elements must be properly nested. Overlapping tags are not allowed (`<b><i>text</i></b>` is correct; `<b><i>text</b></i>` is incorrect).
4. **Closing Tags:** Every opening tag must have a corresponding closing tag, unless it is an empty element which must be self-closing (`<br/>` or `<br></br>`).
5. **Case Sensitivity:** XML is case-sensitive. Opening and closing tags must match exactly (`<Book>` and `</book>` are different).
6. **Attribute Quotes:** All attribute values must be enclosed in either single or double quotation marks.
7. **Unique Attributes:** An element cannot have two attributes with the same name.
8. **Special Characters:** Certain characters (`<`, `>`, `&`, `'`, `"`) must be escaped using entities (`<`, `>`, `&`, `'`, `"`) or placed in CDATA sections.
9. **Valid Names:** Element and attribute names must start with a letter or underscore, not a number or special character (except underscore).
10. **No Spaces in Tags:** There should be no spaces between the opening bracket and the element name (`< book>` is invalid).

Sample Well-Formed XML Document:

```
<?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE library SYSTEM "library.dtd"> <library> <book id="b001"> <title>Data Structures</title> <author>Mark Allen Weiss</author> <year>2018</year> <price>45.00</price> </book> <book id="b002"> <title>Web Technology</title> <author>Robert W. Sebesta</author> <year>2019</year> <price>38.50</price> </book> <book id="b003"> <title>System Analysis</title> <author>Jeffrey L. Whitten</author> <year>2020</year> <price>52.00</price> </book> </library>
```

DTD Schema (library.dtd):

```
<!DOCTYPE library [ <!ELEMENT library (book+)> <!ELEMENT book (title, author, year, price)> <!ATTLIST book id ID #REQUIRED> <!ELEMENT title (#PCDATA)> <!ELEMENT author (#PCDATA)> <!ELEMENT year (#PCDATA)> <!ELEMENT price (#PCDATA)> ]>
```

DTD Explanation:

- `<!ELEMENT library (book+)>` — The library element must contain one or more book elements.
- `<!ELEMENT book (title, author, year, price)>` — Each book element must contain exactly one title, author, year, and price in that order.
- `<!ATTLIST book id ID #REQUIRED>` — The book element must have an id attribute of type ID, which is required and must be unique.
- `<!ELEMENT title (#PCDATA)>` — The title element contains parsed character data (text).
- Similarly, author, year, and price contain #PCDATA.

How to Validate:

1. Save the XML as library.xml and DTD as library.dtd in the same directory.
2. Use an XML parser or validator tool (like XMLSpy, Oxygen XML Editor, or online validators).
3. In PHP, use DOMDocument:

```
<?php $xml = new DOMDocument(); $xml->load('library.xml'); if ($xml->validate()) { echo "XML is valid according to DTD!"; } else { echo "XML is NOT valid!"; } ?>
```

4. In Python, use lxml:

```
from lxml import etree
dtd = etree.DTD(open('library.dtd'))
tree = etree.parse('library.xml')
print(dtd.validate(tree)) # True if valid
```

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.